Informe de seguridad – Sesión 10

Alumno: Tineo · **Curso:** Lenguaje de Programación II (LP2)

Actividad: Reto 02 – Actividad autónoma de la Sesión 10 · **Proyecto:** PharmaBackend (`Practica`), Spring Boot 4, Spring Security 7, JJWT 0.12.6

Última ejecución: 18/10/2026 · Rama `feature/seguridad-pruebas-tineo`

C1. Resumen de la ejecución

Las pruebas se hicieron con Swagger UI sobre una instancia propia (puerto 8081, Oracle en Docker). Cada caso está en `docs/matriz-seguridad-s10.md` con código y mensaje esperados y obtenidos. Resultado: **42 casos ejecutados, 42 pasan, 0 fallan**.

Grupo	IDs	Casos	Pasan	Fallan
Autenticación (A)	A-01 a A-05	5	5	0
Token (T)	T-01 a T-06	6	6	0
Permisos (P)	P-01 a P-05	5	5	0
Usuarios (U)	U-01 a U-03	3	3	0
CORS (C)	C-01	1	1	0
B1 – Cambio de contraseña	B-01 a B-07	7	7	0
B2 – Asignar roles	B-08 a B-13, B-18a	7	7	0
B3 – Activar o desactivar	B-14 a B-17b, B-18b	6	6	0
Restauración de datos de prueba	B-19, B-20	2	2	0
Total		**42**	**42**	**0**

Además hay pruebas unitarias de seguridad que pasan: `JwtServiceTest`, `PermisosPorRolTest`, `AuthServiceImplTest` y `UsuarioServiceImplTest` (esta con los casos 404, 409 y éxito de B2 y B3). Los datos de prueba que se modificaron (contraseña, roles y estado de `vendedor`) se restauraron y se verificaron en Oracle.

C2. Fichas de hallazgos

Ningún caso falló, así que no hay fichas de defectos. Se dejan dos **observaciones**:

OBS-01 · El token sigue válido tras cambiar la contraseña (severidad media)

Descripción: después de `PUT /auth/password` (B-01), el token emitido antes sigue dando 200 en `GET /auth/me` hasta su `exp` (60 minutos). **Evidencia:** B-06 (`B-06_swagger.png`). **Causa probable:** `JwtService.generarToken` solo firma `sub`, `authorities`, `iat` y `exp`, y `JwtAuthenticationFilter.doFilterInternal` no compara el token con nada que cambie al modificar la credencial. **Corrección propuesta:** columna de versión o `passwordChangedAt` en `usuarios`, incluida como claim y verificada en el filtro; o un `jti` con lista de revocación.

OBS-02 · Secreto JWT de desarrollo dentro del repositorio (severidad baja, solo entorno dev)

Descripción: `application-dev.yaml` está versionado y contiene `app.jwt.secret`; el repositorio es público. **Causa probable:** comodidad de desarrollo en `application-dev.yaml`; el perfil de producción (`application-prod.yaml`) sí lo toma de `${JWT_SECRET}`. **Corrección propuesta:** leer también el secreto de desarrollo de una variable de entorno o de un archivo ignorado por git, y rotar el valor actual.

Como contraparte positiva, B-15 demuestra que desactivar una cuenta (B3) sí invalida su token al instante: el filtro recarga al usuario desde la base en cada petición.

C3. Preguntas de análisis

1. ¿Por qué el login responde el mismo mensaje en los tres casos? `GlobalExceptionHandler.handleAuthentication` atiende cualquier `AuthenticationException` (credenciales

malas, usuario inexistente, cuenta deshabilitada) con un único 401 «Usuario o contraseña incorrectos» (casos A-02, A-03 y B-16). Un mensaje distinto permitiría *enumerar usuarios*: el atacante probaría una lista de nombres y sabría cuáles existen y cuáles están deshabilitadas (cuentas olvidadas, objetivos de interés) y dónde solo falta adivinar la contraseña, lo que facilita la fuerza bruta y el relleno de credenciales. Con el mensaje uniforme, los tres casos son indistinguibles desde fuera.

2. ¿Por qué el token anterior sigue válido y cómo invalidarlo? Porque la API no guarda sesión: `JwtService` solo valida la firma y `exp`, y la contraseña no forma parte del token, así que cambiarla no lo toca (B-06). Se puede invalidar con (a) una versión de credenciales o `passwordChangedAt` en `usuarios`, puesta como claim y comprobada en `JwtAuthenticationFilter`, que de todos modos ya consulta al usuario en cada petición; (b) una lista de revocación de `jti` en base de datos o Redis; o (c) tokens de acceso de pocos minutos con un token de refresco que sí se puede revocar.

3. ¿Qué haría un atacante con el secreto publicado y qué hacer de inmediato? Con el secreto puede firmar tokens que `JwtService` acepta (T-03 muestra que un token firmado con otro secreto se rechaza, así que con el correcto sería válido): se haría pasar por `admin`, leería reportes, crearía usuarios y asignaría roles sin conocer ninguna contraseña. De inmediato: (1) generar un secreto nuevo y largo y cargarlo por variable de entorno (`JWT_SECRET`, como ya hace el perfil prod), lo que invalida todos los tokens existentes, también los forjados; (2) reiniciar la aplicación y pedir que se inicie sesión de nuevo; (3) revisar logs y cuentas o roles cambiados; (4) retirar el secreto del repositorio y considerarlo comprometido aunque se borre, porque queda en el historial, y activar el escaneo de secretos.

4. `@PreAuthorize` en `ReporteController` frente a una regla por URL en `SecurityConfig`. Ventaja: la regla (`hasAuthority('REPORTES_VER')`) está junto al código que protege, así que si se agrega un endpoint o cambia una ruta no queda desprotegido por un patrón que ya no coincide, y permite reglas más finas por método. Desventaja: la seguridad queda repartida en dos lugares y `SecurityConfig` ya no muestra el cuadro completo; además requiere `@EnableMethodSecurity` (si se olvida, las anotaciones no hacen nada) y el rechazo ocurre dentro del controlador, por lo que el `AccessDeniedException` debe traducirse a 403 con el manejador global (caso P-03). Por eso el módulo de usuarios, el más crítico, tiene la regla por URL en `SecurityConfig`.